

ORGNZ-06 LAB: Security Context - Hands-on Exercises

Series: ORGNZ | **Notebook:** 6 of 10 | **Type:** LAB | **Created:** February 2026 | **Last Updated:** 02/19/2026

Overview

This lab notebook contains 4 hands-on exercises extracted from **ORGNZ-06: Security Context**. Complete the lecture notebook first, then work through these exercises to reinforce the concepts with real DQL queries against your Dynatrace environment.

Table of Contents

1. [Exercise 1: Why Use Security Context?](#)
 2. [Exercise 2: Why Use Security Context?](#)
 3. [Exercise 3: Why Use Security Context?](#)
 4. [Exercise 4: Why Use Security Context?](#)
 5. [Lab Summary](#)
-

Prerequisites

Requirement	Details
Completed	ORGNZ-06: Security Context (lecture notebook)
Dynatrace Environment	SaaS tenant with Grail enabled
Permissions	logs.read , metrics.read , entities.read , spans.read

Exercise 1: Why Use Security Context?

ORGNZ-06: Security Context

Series: ORGNZ | Notebook: 6 of 10 | Created: January 2026 | Last Updated: 02/05/2026

If permissions on deployment-level attributes or bucket level are insufficient, Dynatrace allows you to set up **fine-grained permissions** by adding a `dt.security_context` attribute to your data. This enables record-level access control that scales beyond bucket limits.

| Requirement | Details |
|---

- 1. What is Security Context?
- 2. Security Context Values
- 3. Setting Security Context
- 4. IAM Policies with Security Context
- 5. Querying Security Context
- 6. Security Context Patterns
- 7. Security Context Best Practices
- 8. Entity Security Context

-----|-----|

| **Dynatrace Account** | Account-level administrative access |

| **Permissions** | OpenPipeline configuration, IAM policy management |

| **Knowledge** | Completed ORGNZ-04 and ORGNZ-05 |

By the end of this notebook, you will:

- Understand the purpose and structure of security context
- Know how to assign security context via OpenPipeline
- Create IAM policies that use security context
- Design hierarchical security context patterns

Security context is a reserved field (`dt.security_context`) that enables custom authorization:

Aspect	Description
Field name	<code>dt.security_context</code>
Type	String or array of strings
Purpose	Custom authorization beyond standard attributes
Source	OpenPipeline, OneAgent, Kubernetes labels
Enforcement	IAM policies filter at query time

Scenario	Without Security Context	With Security Context
100 teams sharing data	Need 100 buckets	Single bucket, 100 context values
Dynamic team membership	Static bucket assignment	Update context via tags
Hierarchical access	Complex policy combinations	Hierarchical string encoding

```
dt.security_context: "team-a"  
dt.security_context: "finance"  
dt.security_context: "sec-lvl-7"
```

Encode hierarchy in a string for flexible access patterns:

```
dt.security_context: "department-A/team-1/project-x"
dt.security_context: "org1/finance/compliance"
dt.security_context: "region-us/aws-account-123/team-platform"
```

Records can have multiple security context values:

```
dt.security_context: ["team-a", "project-x", "compliance"]
```

Configure OpenPipeline to add security context based on record attributes:

1. Go to **Settings > Log Processing > OpenPipeline**
2. Select your pipeline
3. Go to **Permission** tab
4. Add **Set Security Context** processor

```
# OpenPipeline security context processor
processors:
  - type: security-context
    rules:
      - condition: "host.group starts-with 'finance-'"
        context: "lob:finance"
      - condition: "k8s.namespace.name == 'checkout'"
        context: "team:checkout"
      - condition: "matchesValue(service.name, 'payment-*')"
        context: "team:payments"
```

OneAgent can set security context based on:

Source	Configuration
Host groups	Automatic from host group membership
Kubernetes labels	Map labels to security context
Cloud metadata	Use cloud account/project info
Custom tags	Reference existing tags

Use labels or annotations to set security context:

```
metadata:
  labels:
    dynatrace.com/security-context: "team:checkout"
  annotations:
    dynatrace.com/security-context: "compliance:pci"
```

```
{
  "name": "team-a-data-access",
  "description": "Team A can access data with team-a security context",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name
STARTSWITH 'default_'; ALLOW storage:logs:read WHERE
storage:dt.security_context = 'team-a';",
  "tags": ["team:team-a"]
}
```

Grant access to entire department:

```
{
  "name": "finance-department-access",
  "description": "Finance department can access all finance sub-teams",
  "statementQuery": "ALLOW storage:buckets:read WHERE storage:bucket-name
STARTSWITH 'default_'; ALLOW storage:logs:read WHERE
storage:dt.security_context MATCH ('finance/*');",
  "tags": ["department:finance"]
}
```

Important: When `dt.security_context` holds an array, you MUST use `MATCH` operator. Using `=`, `STARTSWITH`, or `IN` will always return false for array fields.

```
// Correct - MATCH for array fields
ALLOW storage:logs:read WHERE storage:dt.security_context MATCH ("crn-70400-
*");

// Wrong - = won't work for arrays
ALLOW storage:logs:read WHERE storage:dt.security_context = "crn-70400-team";
```

```
// View logs with security context
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| fields timestamp, content, dt.security_context
| limit 20
```

Exercise 2: Why Use Security Context?

```
// Summarize data by security context
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| summarize recordCount = count(), by:{dt.security_context}
| sort recordCount desc
| limit 20
```

Exercise 3: Why Use Security Context?

```
// Filter logs by specific security context
fetch logs, from:-1h
| filter dt.security_context == "team:platform"
| limit 50
```

Exercise 4: Why Use Security Context?

```
// Check for hierarchical security context patterns
fetch logs, from:-1h
| filter isNotNull(dt.security_context)
| filter startsWith(dt.security_context, "finance/")
| summarize count = count(), by:{dt.security_context}
| sort count desc
```

Lab Summary

You have completed 4 hands-on exercises for **ORGNZ-06: Security Context**.

Exercises Completed

- ☐ Exercise 1: Why Use Security Context?
- ☐ Exercise 2: Why Use Security Context?
- ☐ Exercise 3: Why Use Security Context?

- [] Exercise 4: Why Use Security Context?

Next Steps

Continue with **ORGNZ-07** for the next notebook.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.